Programacion dinamica

Que es?

- Paradigma de resolucion de problemas
- Similar en algunos aspectos a divide y venceras y backtracking
- Resumen de divide y venceras:
  - Dividir el problema en subproblemas *independientes*
  - Resolver cada subproblema recursivamente
  - Combinar las soluciones en una solucion del problema original
- Programacion dinamica:
  - Dividir el problema en subproblemas *que se solapan*
  - Resolver cada subproblema recursivamente
  - Combinar las soluciones en una solucion del problema original
  - No calcular la respuesta de el mismo subproblema mas de una vez

Formulacion de programacion dinamica

1. Formular el problema en terminos mas pequeños del mismo problema
2. Convertir el problema formulado en una funcion recursiva
3. *Memoizar* la funcion (recordar los resultados que ya han sido calculados)

Formulacion de divide and conquer

```
valor dc(problema P) {
   if ( es_caso_base(P) ) {
      return valor_de_caso_base(P);
   }

   valor resultado = algun valor;
   for ( problema Q en subproblemas(P) ) {
      resultado = combinar( resultado, dc(Q) );
   }

   return resultado;
}
```

Formulacion de programacion dinamica

```
map<problema, valor> memoria;

valor dp(problema P) {
   if ( es_caso_base(P) ) {
      return valor_de_caso_base(P);
   }

   if ( memoria.find(P) != memoria.end() ) {
      return memoria[P];
   }

   valor resultado = algun valor;
   for ( problema Q en subproblemas(P) ) {
      resultado = combinar( resultado, dp(Q) );
   }

   memoria[P] = resultado;
   return resultado;
}
```

Fibonacci

Los primeros dos numeros en la secuencia de Fibonacci son 1 y 1. Todos los otros numeros estan definidos como la suma de los anteriores dos numeros en la secuencia.

- Tarea: Enncontrar el n-esimo numero en la secuencia
- Vamos a resolverlo con programacion dinamica!

1. Formular el problema en terminos mas pequeños del mismo problema

fibonacci(1) = 1

fibonacci(2) = 1

fibonacci(n) = fibonacci(n - 2) + fibonacci(n - 1)

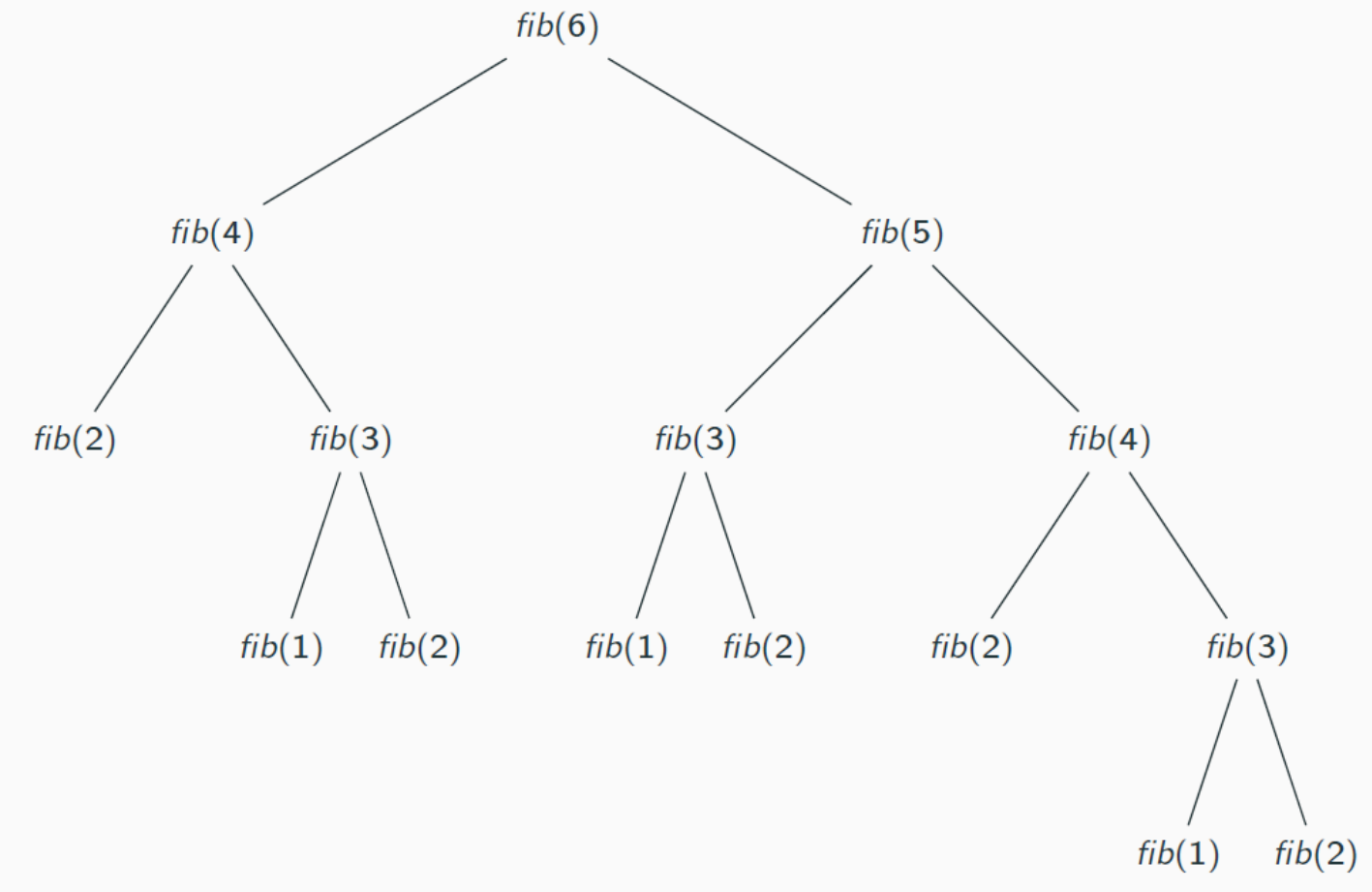
Fibonacci

2. Convertir el problema formulado en una funcion recursiva

```
int fibonacci(int n) {
    if (n <= 2) {
        return 1;
    }

    int res = fibonacci(n - 2) + fibonacci(n - 1);

    return res;
}
```

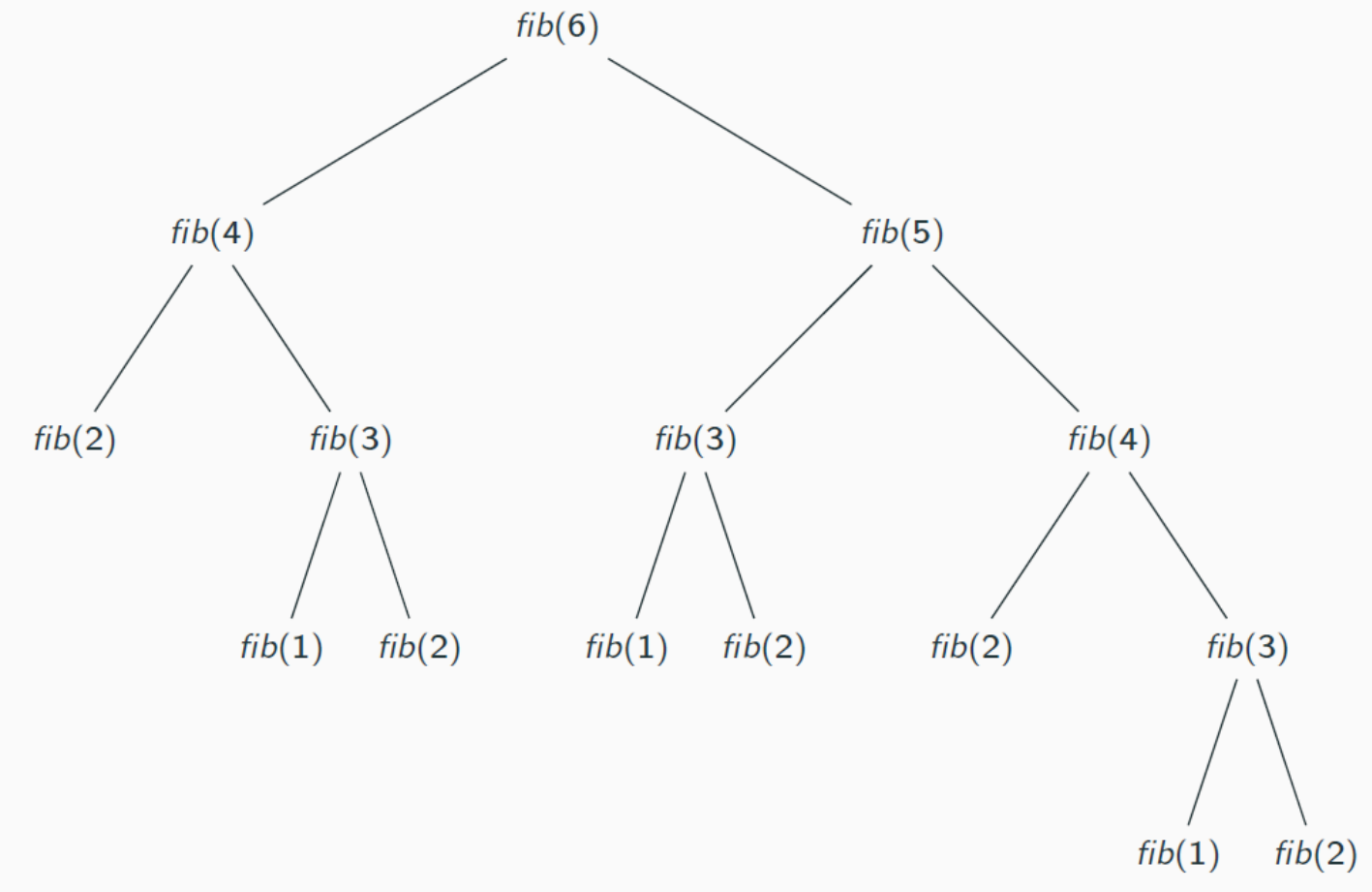
Fibonacci

Cual es la complejidad?



Fibonacci

Cual es la complejidad? Exponencial, casi $O(2^n)$



Fibonacci

3. Memoizar la funcion (recordar los resultados que ya han sido calculados)

```
map<int, int> mem;

int fibonacci(int n) {
    if (n <= 2) {
        return 1;
    }

    if (mem.find(n) != mem.end()) {
        return mem[n];
    }

    int res = fibonacci(n - 2) + fibonacci(n - 1);

    mem[n] = res;
    return res;
}
```

Fibonacci

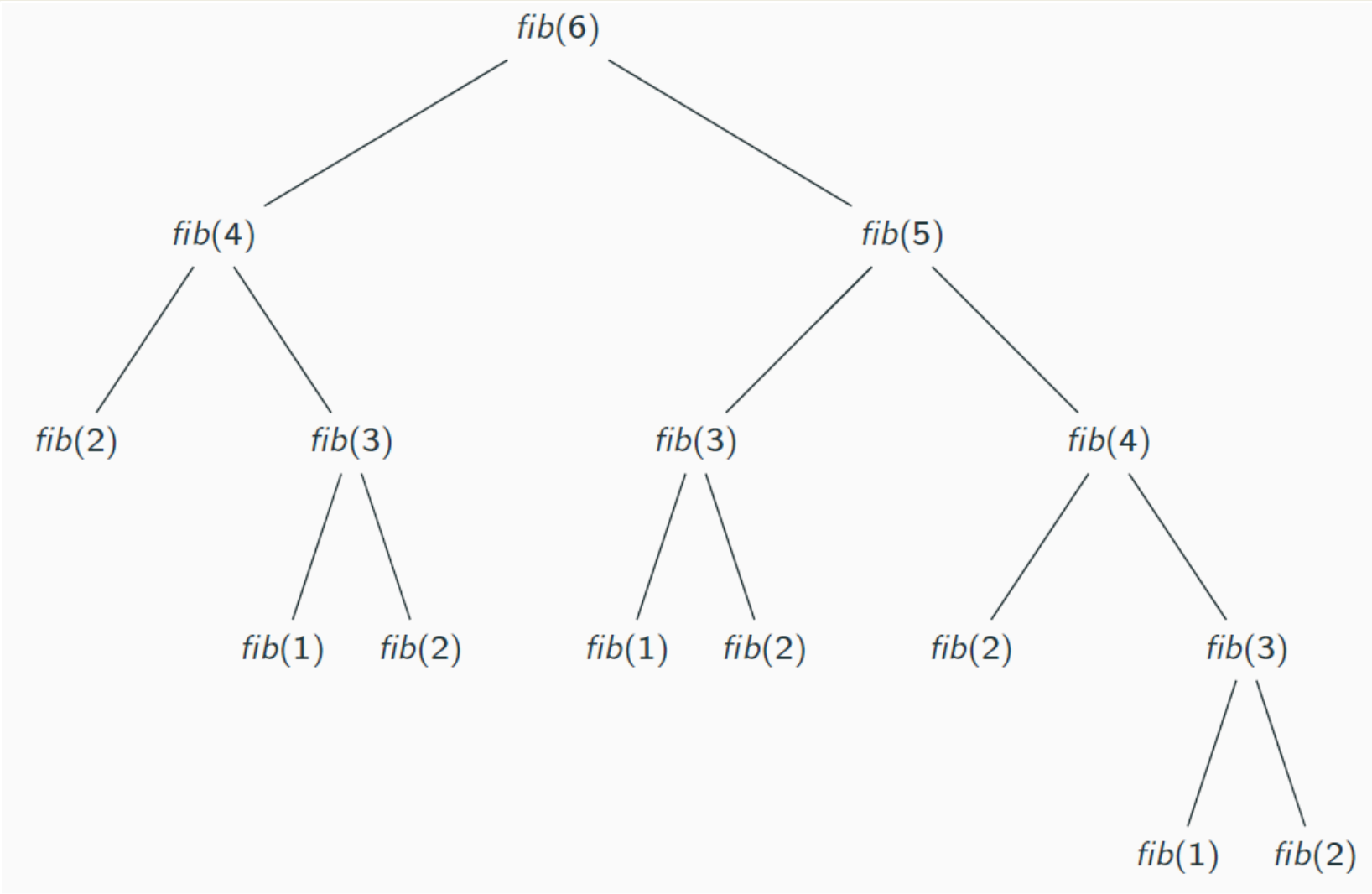
```
int mem[1000];
for (int i = 0; i < 1000; i++)
    mem[i] = -1;

int fibonacci(int n) {
    if (n <= 2) {
        return 1;
    }

    if (mem[n] != -1) {
        return mem[n];
    }

    int res = fibonacci(n - 2) + fibonacci(n - 1);

    mem[n] = res;
    return res;
}
```

Fibonacci

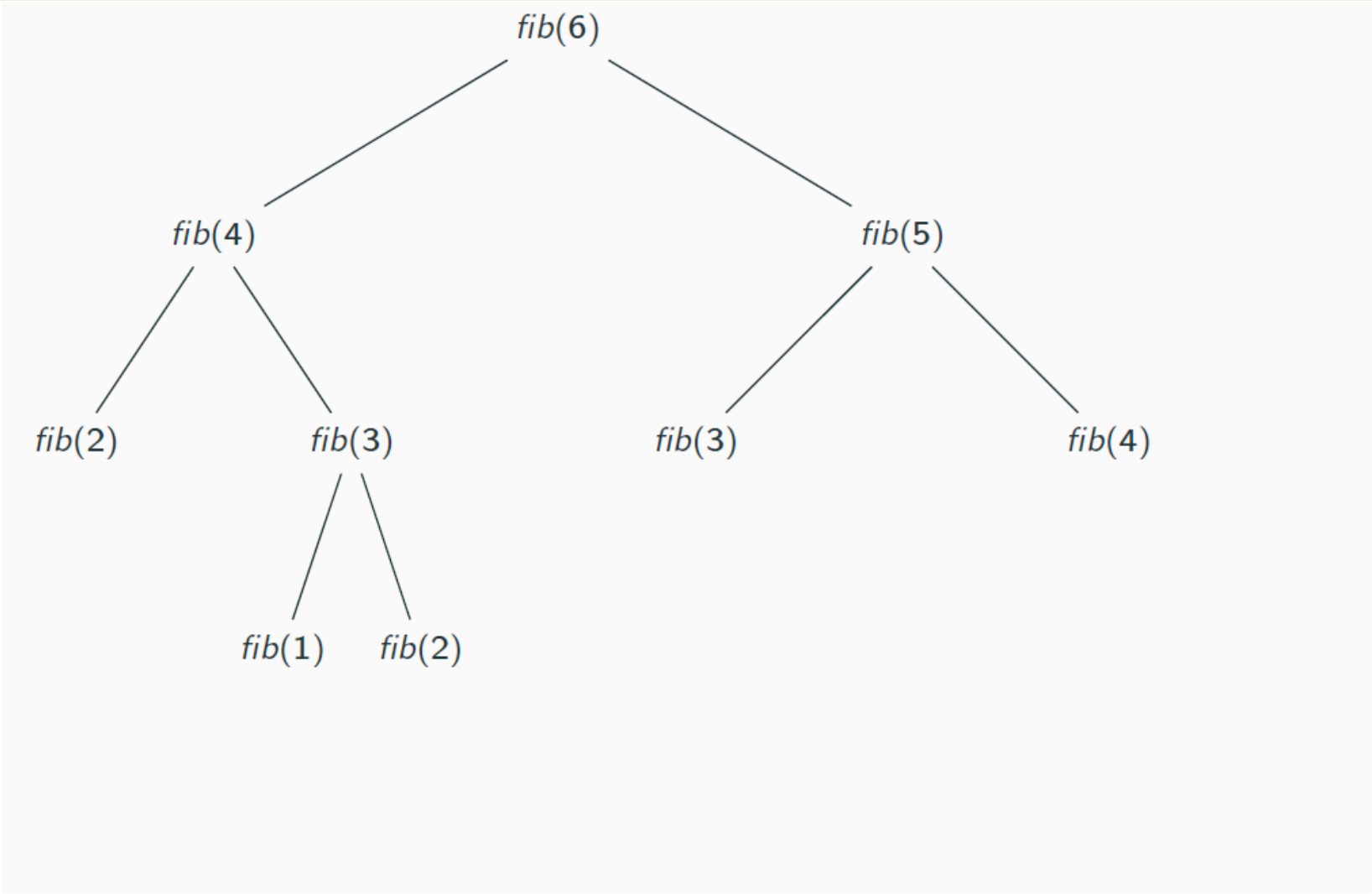
- Cual es la complejidad ahora?

- Hay n posibles entradas para la funcion: 1, 2, …, n
- Cada entrada hara una de las dos cosas:
  - Ser calculada, y el resultado se guarda
  - Se devolvera la solucion desde la memoria
- Cada entrada sera calculada a lo mucho una vez
- La complejidad es O(n x f), donde f es la complejidad de calcular una entrada si asumimos que los subproblemas ya han sido resueltos (O(1))
- Ya que hacemos operaciones simples para calcular la respuest de una entrada, f = O(1)
- La complejidad de tiempo total es O(n)

Fibonacci


fib(6)
fib(4)
fib(5)
fib(2)
fib(3)
fib(3)
fib(4)
fib(1)
fib(2)
fib(1)
fib(2)
fib(2)
fib(3)
fib(1)
fib(2)

Fibonacci



Suma maxima

- Dado un arreglo arr[0], arr[1], …, arr[n - 1] de enteros, encontrar el intevalo con la maxima suma

-15	8	-2	1	0	6	-3

- La suma maxima de un intevalo en el arreglo es 13
- Como resolvemos este problema en general?
  - Es facil iterar por todos los ≈ n^2 intervalos y calcular sus sumas, pero eso tomaria $O(n^3)$
  - Podriamos usar una tecnica de suma estatica en rangos para bajar el tiempo a $O(n^2)$
  - Podemos hacerlo mejor con programacion dinamica?

Suma maxima

- Primero debemos formular el problema recursivamente
- Decimos que max_sum(i) es la maxima suma de un intervalo en el rango 0, …, i
- Caso base: max_sum(0) = max(0, arr[0])
- Que hay de max_sum(i)?
- Que retorna max_sum(i - 1)?
- Es posible combinar las soluciones de subproblemas con un *i* menor a la solucion para *i*?

Suma maxima

- Intenemos hacer otra cosa
- Que max_sum(i) sea la suma maxima de un intervalo en el rango 0, …, i que *termina* en i
- Caso base: max_sum(0) = arr[0]
- Max_sum(i) = max(arr[i], arr[i] + max_sum(i - 1))

- Entonces la respuesta seria el maximo *max_sum(i)* para $0 \leq i < n$

Suma maxima

- El siguiente paso es volver esto en una funcion

```
int arr[1000];

int max_sum(int i) {
    if (i == 0) {
        return arr[i];
    }

    int res = max(arr[i], arr[i] + max_sum(i - 1));

    return res;
}
```

Suma maxima

- El ultimo paso es memoizar la funcion

```
int arr[1000];
int mem[1000];
bool comp[1000];
memset(comp, 0, sizeof(comp));

int max_sum(int i) {
    if (i == 0) {
        return arr[i];
    }
    if (comp[i]) {
        return mem[i];
    }

    int res = max(arr[i], arr[i] + max_sum(i - 1));

    mem[i] = res;
    comp[i] = true;
    return res;
}
```

Suma maxima

- Entonces la respuesta es solo el maximo de todos los finales de intervalos

```
int maximum = 0;
for (int i = 0; i < n; i++) {
    maximum = max(maximum, best_sum(i));
}

printf("%d\n", maximum);
```

Suma maxima

- Que hay de la complejidad?
- Hay *n* posibles entradas para la funcion
- Cada entrada es procesada en O(1) suponiendo que los subproblemas ya han sido resueltos (O(1))
- La complejidad es O(n)

Coin change

- Dados los valores de una moneda m_0, $m_{1,}$ …, m_{n-1} y una cantidad de dinero x: Cual es el minimo numero de monedas necesario que sumen x?
- Recuerdan el algorimo greedy para Coin Change?
- No siempre daba una solucion optima, y algunas veces ni siquiera llegaba a una solucion…
- Y si usamos programacion dinamica?

Coin change

- Primero: formular el problema de forma recursiva

- Que opt(i, x) sea el minimo numero de monedas necesarias que sumen x si solo usamos monedas $m_{0,}$ …, m_i

- Caso base: opt(i, x) = ∞ si x < 0
- Caso base: opt(i, 0) = 0
- Caso base: (opt(-1, x) = ∞

- opt(I, x) = $\min\begin{cases} 1+\mathrm{opt}(i, x - m_i) \\ \mathrm{opt}(i - 1, x) \end{cases}$

Coin change

Digamos que hay 10 tipos de monedas

```
int INF = 100000;
int d[10];

int opt(int i, int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (i == -1) return INF;

    int res = INF;
    res = min(res, 1 + opt(i, x - d[i]));
    res = min(res, opt(i - 1, x));

    return res;
}
```

Coin change

```
int INF = 100000;
int d[10];
int mem[10][10000];
memset(mem, -1, sizeof(mem));

int opt(int i, int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (i == -1) return INF;

    if (mem[i][x] != -1) return mem[i][x];

    int res = INF;
    res = min(res, 1 + opt(i, x - d[i]));
    res = min(res, opt(i - 1, x));

    mem[i][x] = res;
    return res;
}
```

Coin change

- Cual es la complejidad?
- El numero de posibles entradas es $n * x$
- Cada entrada sera procesada en O(1) asumiendo que se resolvieron los subproblemas
- Complejidad total: O(n * x)

Longest increasing subsequence

- Dado un arreglo a[0], a[1], …, a[n - 1] de enteros, cual es el tamaño de la subsecuencia creciente mas larga?
- Primero, que es una subsecuencia?
- Si eliminamos cero o mas elementos de *a*, entonces tenemos una subsecuencia de *a*
- Ejemplo: a = [5, 1, 8, 1, 9, 2]
- [5, 8, 9] es una subsecuencia
- [1, 1] es una subsecuencia
- [5, 1, 8, 1, 9, 2] es una subsecuencia
- [] es una subsecuencia
- [8, 5] **no** es una subsecuencia
- [10] **no** es una subsecuencia

Longest increasing subsequence

- Una subsecuencia creciente es una subsecuencia de *a* tal que los elementos en ella estan en un orden *estrictamente* creciente
- [5, 8, 9] y [1, 8, 9] son las subsecuencias crecientes mas largas de a = [5, 1, 8, 1, 9, 2]

- Como calculamos el tamaño de la subsecuencia comun mas larga?
- Hay 2^n subsecuencias, podriamos revisarlas todas
- Eso tomaria $O(n2^n)$, que terminaria rapidamente para $n \leq 23$
- Y si usamos programacion dinamica?

Longest increasing subsequence

- Digamos que lis(i) es la longitud de la secuencia creciente mas larga del arreglo a[0], …, a[i]
- Caso base: lis(0) = 1
- Que hay de lis(i)?
- Tenemos el mismo problema que en el de maxima suma, hay que cambiar de perspectiva

Longest increasing subsequence

- Que lis(i) sea la longitud de la subsecuencia creciente mas larga del arreglo a[0], …, a[i] que *termina en i*
- Caso base: no necesitamos uno
- lis(i) = max(1, 1 + lis(j)) para j < i tal que a[j] < a[i]

Longest increasing subsequence

```
int a[1000];
int mem[1000];
memset(mem, -1, sizeof(mem));

int lis(int i) {
    if (mem[i] != -1) {
        return mem[i];
    }

    int res = 1;
    for (int j = 0; j < i; j++) {
        if (a[j] < a[i]) {
            res = max(res, 1 + lis(j));
        }
    }

    mem[i] = res;
    return res;
}
```

Longest increasing subsequence

- Entonces para hallar la subsecuencia comun mas larga de todo el arreglo, solo revisamos todos los puntos finales

```
int mx = 0;
for (int i = 0; i < n; i++) {
    mx = max(mx, lis(i));
}

printf("%d\n", mx);
```

Longest increasing subsequence

- La complejidad?

- Hay n posibles entradas
- Cada entrada es calculada en O(n), asumiendo que los subproblemas han sido resueltos
- La complejidad de tiempo total es $O(n^2)$

- Esto es lo suficientemente rapido para $n \leq 10\,000$, mucho mejor que el metodo con fuerza bruta!

top-down / bottom-up

UVa 10465 - Homer Simpson

El problema nos dice que en Springfield hay dos tipos de hamburguesas, las Krusty-burger y las de Kwik-e-Mart, a Homero le toma *n* minutos comer una y *m* minutos la otra. Homero tiene *t* minutos para comer, a el le encanta comer asi que quiere minimizar el tiempo perdido si come una hamburguesa tras otra. Dile a Homero cual es el maximo numero de hamburguesas que puede comer perdiendo el menor tiempo posible, y si piede tiempo, cuanto.

$n, m, t \leq 10\,000$

ej: para $n = 3$, $m = 5$, $t = 54$ Homero come 18 hamburguesas

para $n = 3$, $m = 5$, $t = 55$ come 17

En ambos casos no pierde tiempo

Como resolvemos este problema?

UVa 10465 - Homer Simpson

- No podemos hacer busqueda completa, si n y m < 10 y t > 9000 ver todas las posibilidades tomaria muchisimo tiempo

- Un algoritmo greedy podria ser siempre comer la hamburguesa que tome menos tiempo, pero no sabriamos en que punto empezar a tomar la otra.

- Y programacion dinamica?
- El problema no nos permite empezar desde el final ya que no siempre Se usara todo el tiempo (ej: n = 2, m = 2, t = 5) por lo que pensar en una solucion top down es complicado (ejercicio para ustedes… :D).

- Podemos hacer bottom up

UVa 10465 - Homer Simpson

- Vemos que el problema se reduce a lo siguiente: hasta cierto tiempo *i*, cual es el numero maximo de hamburguesas que se puede comer.
- Entonces, tenemos que calcular ese numero para cada tiempo y luego la respuesta estaria en el mayor tiempo al que pudimos llegar.
- Como calculamos esto?
- Partimos de que en el tiempo 0, Homero no puede comer ninguna hamburguesa.
- Vamos avanzando en el tiempo y a cada paso preguntamos: cual es el maximo numero de hamburguesas que puede comer en el tiempo $i - n$? cual es el numero en $i - m$? Elegimos el mayor y le sumamos 1.
- Esto lo podemos hacer con un recorrido desde 1 hasta *t*, por lo que la complejidad es O(t)

Uva 10943 - How do you add?

Resumen: Dados los numeros n y k, calcular cuantas posibles formas hay de sumar n usando k numeros entre 0 y n

$n, k \leq 100$

- Que nos interesa en cada estado? Cuales son los subproblemas que se solapan?

Uva 10943 - How do you add?

Resumen: Dados los numeros *n* y *k*, calcular cuantas posibles formas hay de sumar *n* usando *k* numeros entre 0 y *n*

$n, k \leq 100$

- Que nos interesa en cada estado? Cuales son los subproblemas que se solapan?

- Si hacemos la solucion con busqueda completa, nos damos cuenta de que nos interesa saber cuantas posibles formas hay de sumar *n'* usando *k'* numeros ($n' \leq n$, $k' \leq k$).

- Hay *n* x *k* posibles entradas, y cada entrada se calcula en O(n)
- La complejidad total es $O(n^2 \times k)$

Dejare como ejercicio para ustedes encontrar la solucion bottom up :)